

✓ Exercice 2 : Transfer Learning — Chat vs Chien

TP2 — Réseaux de Neurones Convolutifs

Objectif : Utiliser le Transfer Learning pour classifier des images de chats et chiens.

Pourquoi ? Entraîner un CNN from scratch nécessite beaucoup de données et de temps. Le Transfer Learning réutilise un modèle déjà entraîné sur ImageNet (millions d'images).

✓ Étape 1 — Activer le GPU

Runtime → Change runtime type → GPU

```
import tensorflow as tf
print(f"GPU disponible : {tf.config.list_physical_devices('GPU')}")
```

```
GPU disponible : [PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')]
```

✓ Étape 2 — Imports

```
print('Mise à jour de tensorflow-datasets et protobuf...')
!pip install --upgrade tensorflow-datasets protobuf
print('Mise à jour terminée. Veuillez redémarrer l\'environnement d\'exécution (Runtime -> Redémarrer l\'environne
```

```
Mise à jour de tensorflow-datasets et protobuf...
Requirement already satisfied: tensorflow-datasets in /usr/local/lib/python3.12/dist-packages (4.9.10)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (7.35.1)
```

Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (1.4.0)
Requirement already satisfied: array_record>=0.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow-data)
Requirement already satisfied: dm-tree in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.1.1)
Requirement already satisfied: etils>=1.9.1 in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etre])
Requirement already satisfied: immutabledict in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.0.2)
Requirement already satisfied: promise in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.3)
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (5.9.5)
Requirement already satisfied: pyarrow in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (18.1.0)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow-dataset)
Requirement already satisfied: simple_parsing in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets)
Requirement already satisfied: tensorflow-metadata in /usr/local/lib/python3.12/dist-packages (from tensorflow-data)
Requirement already satisfied: termcolor in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (3.3.0)
Requirement already satisfied: toml in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (0.10.2)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (4.67.3)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from tensorflow-datasets) (2.2.2)
Requirement already satisfied: einops in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etre])
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etre])
Requirement already satisfied: typing_extensions in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etre])
Requirement already satisfied: zipp in /usr/local/lib/python3.12/dist-packages (from etils[edc,enp,epath,epy,etre])
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->tensorflow-datasets)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->tensorflow-datasets)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->tensorflow-datasets)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->tensorflow-datasets)
Requirement already satisfied: attrs>=18.2.0 in /usr/local/lib/python3.12/dist-packages (from dm-tree->tensorflow-datasets)
Requirement already satisfied: six in /usr/local/lib/python3.12/dist-packages (from promise->tensorflow-datasets) (1.16.0)
Requirement already satisfied: docstring-parser~0.15 in /usr/local/lib/python3.12/dist-packages (from simple_parsing->tensorflow-datasets)
Requirement already satisfied: googleapis-common-protos<2,>=1.56.4 in /usr/local/lib/python3.12/dist-packages (from tensorflow-metadata->tensorflow-datasets)
 Mise à jour terminée. Veuillez redémarrer l'environnement d'exécution (Runtime -> Redémarrer l'environnement d'exécution)

```
import tensorflow_datasets as tfds
import matplotlib.pyplot as plt
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.callbacks import EarlyStopping

print("Prêt !")
```

✓ Étape 3 — Charger le dataset Cats vs Dogs

Dataset :

- 23 262 images couleur
- 2 classes : Chat (0) / Chien (1)
- Tailles variées → redimensionnement en 224×224

```
# Charger le dataset
try:
    (train_ds, val_ds, test_ds), info = tfds.load(
        'cats_vs_dogs',
        split=['train[:80%]', 'train[80%:90%]', 'train[90%:]'],
        with_info=True,
        as_supervised=True
    )
    print(f"Info : {info}")
except AttributeError as e:
    print(f"Erreur: {e}")
    print("Veuillez essayer de redémarrer l'environnement d'exécution (Runtime -> Redémarrer l'environnement d'exé")
except Exception as e:
    print(f"Une erreur inattendue est survenue lors du chargement du dataset: {e}")
```


WARNING:absl:Variant folder /root/tensorflow_datasets/cats_vs_dogs/4.0.1 has no dataset_info.json
Downloading and preparing dataset Unknown size (download: Unknown size, generated: Unknown size, total: Unknown size)
DL Completed : 100% 1/1 [00:16<00:00 16.02s/ url]

```
# Afficher quelques images
plt.figure(figsize=(12, 5))
for i, (image, label) in enumerate(train_ds.take(10)):
    plt.subplot(2, 5, i+1)
    plt.imshow(image)
    plt.title('Chien' if label.numpy() == 1 else 'Chat')
    plt.axis('off')
plt.tight_layout()
plt.show()
```

homepage= <https://www.microsoft.com/en-us/download/details.aspx?id=54765>.



```
publisher = {Association for Computing Machinery, Inc.},  
url = {https://www.microsoft.com/en-us/research/publication/asirra-a-captcha-that-exploits-interest-aligned-man},  
edition = {Proceedings of 14th ACM Conference on Computer and Communications Security (CCS)},  
}""",  
)
```

✓ Étape 4 — Prétraitement

```
import tensorflow as tf

# Taille cible
IMG_SIZE = 224

# Fonction de prétraitement
def preprocess(image, label):
    image = tf.image.resize(image, (IMG_SIZE, IMG_SIZE))
    image = image / 255.0 # Normalisation
    return image, label

# Appliquer
train_ds = train_ds.map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)
val_ds = val_ds.map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)
test_ds = test_ds.map(preprocess).batch(32).prefetch(tf.data.AUTOTUNE)

print("Données prétraitées !")
```

Données prétraitées !

✓ Étape 5 — Data Augmentation

Pour éviter le surapprentissage, on augmente artificiellement le dataset en modifiant les images (rotation, flip, zoom...).

```
data_augmentation = Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.1),
])
```

✓ Étape 6 — Construire le modèle avec Transfer Learning

Principe :

1. Charger MobileNetV2 pré-entraîné sur ImageNet
2. Geler les couches convolutionnelles (features préservées)
3. Ajouter un classifieur binaire (chat/chien)

```
# Charger MobileNetV2 sans la couche de classification
base_model = tf.keras.applications.MobileNetV2(
    weights='imagenet',
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

# Geler le modèle de base
base_model.trainable = False

# Construire le modèle complet
model = Sequential([
    data_augmentation,
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()
```


Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weight_9406464/9406464 2s 0us/step

Model: "sequential_1"

Layer (type)	Output Shape	Param #
sequential (Sequential)	?	0 (unbuilt)
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	?	0
dropout (Dropout)	?	0
dense (Dense)	?	0 (unbuilt)

Total params: 2,257,984 (8.61 MB)

Trainable params: 0 (0.00 B)

Non-trainable params: 2,257,984 (8.61 MB)

✓ Étape 7 — Entraîner

```
# Early stopping
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=3,
    restore_best_weights=True
)

# Entraîner
history = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
```

```
callbacks=[early_stop]
)
```

```
Epoch 1/10
582/582 ————— 60s 87ms/step - accuracy: 0.9483 - loss: 0.1324 - val_accuracy: 0.9798 - val_loss: 0.0
Epoch 2/10
582/582 ————— 41s 69ms/step - accuracy: 0.9619 - loss: 0.0912 - val_accuracy: 0.9811 - val_loss: 0.0
Epoch 3/10
582/582 ————— 41s 70ms/step - accuracy: 0.9653 - loss: 0.0879 - val_accuracy: 0.9819 - val_loss: 0.0
Epoch 4/10
582/582 ————— 40s 69ms/step - accuracy: 0.9673 - loss: 0.0849 - val_accuracy: 0.9798 - val_loss: 0.0
Epoch 5/10
582/582 ————— 41s 70ms/step - accuracy: 0.9669 - loss: 0.0863 - val_accuracy: 0.9828 - val_loss: 0.0
Epoch 6/10
582/582 ————— 40s 69ms/step - accuracy: 0.9684 - loss: 0.0808 - val_accuracy: 0.9815 - val_loss: 0.0
Epoch 7/10
582/582 ————— 40s 69ms/step - accuracy: 0.9670 - loss: 0.0850 - val_accuracy: 0.9819 - val_loss: 0.0
Epoch 8/10
582/582 ————— 41s 69ms/step - accuracy: 0.9668 - loss: 0.0841 - val_accuracy: 0.9819 - val_loss: 0.0
Epoch 9/10
582/582 ————— 42s 72ms/step - accuracy: 0.9684 - loss: 0.0801 - val_accuracy: 0.9815 - val_loss: 0.0
```

✓ Étape 8 — Évaluer

```
# Évaluation
loss, acc = model.evaluate(test_ds)
print(f"\nTest accuracy : {acc:.2%}")
```

```
73/73 ————— 4s 55ms/step - accuracy: 0.9867 - loss: 0.0359
```

```
Test accuracy : 98.67%
```

✓ Étape 9 — Courbes d'apprentissage

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```

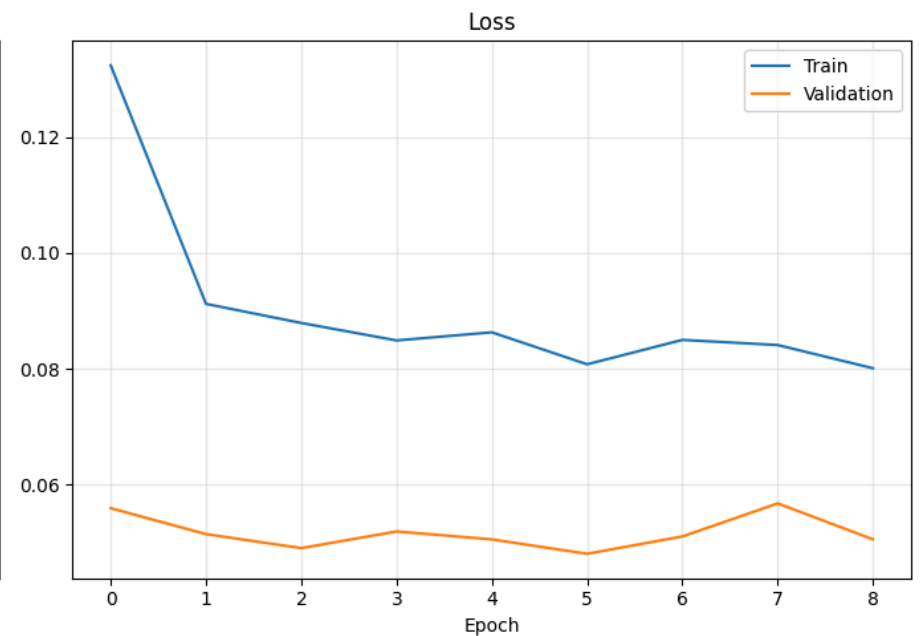
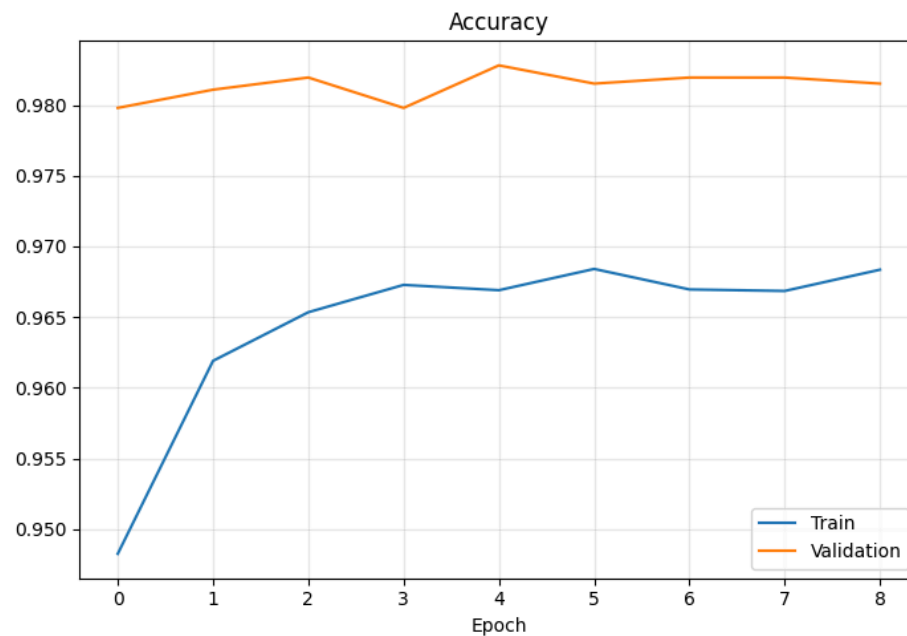
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(history.history['accuracy'], label='Train')
axes[0].plot(history.history['val_accuracy'], label='Validation')
axes[0].set_title('Accuracy')
axes[0].set_xlabel('Epoch')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(history.history['loss'], label='Train')
axes[1].plot(history.history['val_loss'], label='Validation')
axes[1].set_title('Loss')
axes[1].set_xlabel('Epoch')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```



✓ Évaluation détaillée (Précision, Rappel, F1-score, ROC, AUC)

```
from sklearn.metrics import classification_report, roc_curve, auc
import numpy as np

# Obtenir les vraies étiquettes et les prédictions
y_true = []
y_pred_proba = []

for images, labels in test_ds:
    y_true.extend(labels.numpy())
    y_pred_proba.extend(model.predict(images, verbose=0).flatten())

y_true = np.array(y_true)
y_pred_proba = np.array(y_pred_proba)

# Convertir les probabilités en prédictions binaires (seuil 0.5)
y_pred = (y_pred_proba > 0.5).astype(int)

print("\n--- Rapport de Classification ---")
print(classification_report(y_true, y_pred, target_names=['Chat', 'Chien']))

# Calculer la courbe ROC et l'AUC
fpr, tpr, thresholds = roc_curve(y_true, y_pred_proba)
roc_auc = auc(fpr, tpr)

print(f"AUC: {roc_auc:.4f}")
```

```
--- Rapport de Classification ---
              precision    recall  f1-score   support

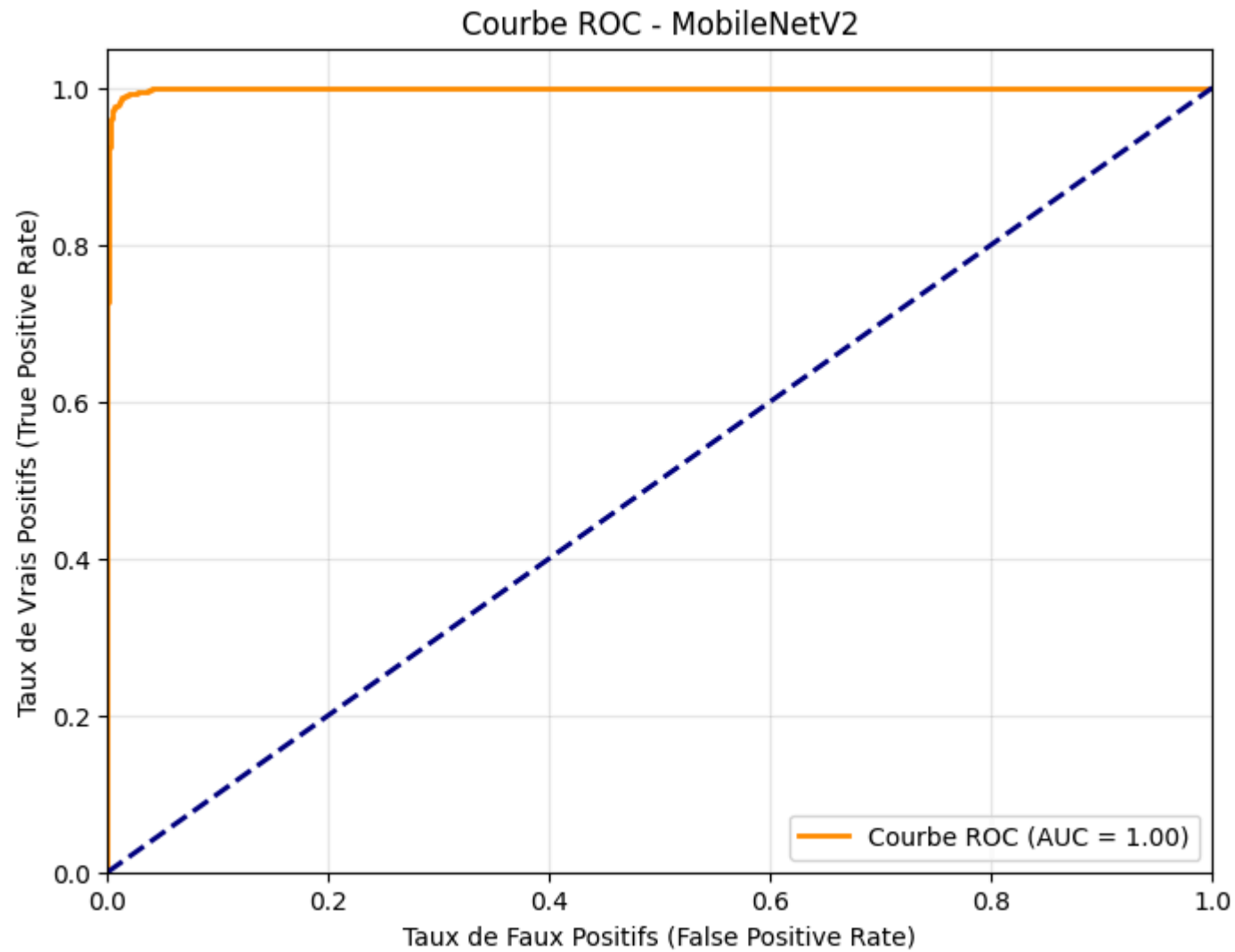
     Chat         0.99        0.99        0.99        1160
     Chien         0.99        0.99        0.99        1166

 accuracy                   0.99         2326
 macro avg              0.99        0.99        0.99         2326
```

weighted avg	0.99	0.99	0.99	2326
--------------	------	------	------	------

AUC: 0.9991

```
fig = plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'Courbe ROC (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Taux de Faux Positifs (False Positive Rate)')
plt.ylabel('Taux de Vrais Positifs (True Positive Rate)')
plt.title('Courbe ROC - MobileNetV2')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.show()
```



✓ step_artifacts

```
# @title step_artifacts
num_fig = "3" # @param {type:"string"}
step = 'ModelEvaluation' # @param ["DataLoading", "DataPreparation", "ModelTraining", "ModelEvaluation"]
```

num_fig

3

step

ModelEvaluation ▼

```
# Assuming `fig` is the current matplotlib figure object
# upload_plt_to_gcs(num_fig, step, fig)
```

[Masquer le code](#)

Résumé

Concept	Description
Transfer Learning	Réutiliser un modèle pré-entraîné
ImageNet	Dataset de 14M d'images, 1000 classes
MobileNetV2	CNN léger, optimisé pour la vitesse
Data Augmentation	Augmenter la diversité du dataset
Binary Crossentropy	Loss pour la classification binaire

Pourquoi c'est efficace ?

- Les features basses (contours, textures) sont universelles
- Seul le classifieur est entraîné → rapide

✓ Implémentation et Évaluation de VGG16

✓ Construction du modèle VGG16

```
from tensorflow.keras.applications import VGG16

# Charger VGG16 sans la couche de classification
base_model_vgg16 = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

# Geler le modèle de base VGG16
```

```
base_model_vgg16.trainable = False

# Construire le modèle complet avec VGG16
model_vgg16 = Sequential([
    data_augmentation, # Utiliser la même augmentation de données
    base_model_vgg16,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])

model_vgg16.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

print("Modèle VGG16 construit et compilé :")
model_vgg16.summary()
```


Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_order_58889256/58889256 4s 0us/step

Modèle VGG16 construit et compilé :

Model: "sequential_2"

Layer (type)	Output Shape	Param #
sequential (Sequential)	(None, 224, 224, 3)	0
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)	0
dropout_1 (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 1)	513

Total params: 14,715,201 (56.13 MB)

Trainable params: 513 (2.00 KB)

Non-trainable params: 14,714,688 (56.13 MB)

✓ Entraînement du modèle VGG16

```
# Entraîner VGG16
history_vgg16 = model_vgg16.fit(
    train_ds,
    validation_data=val_ds,
    epochs=10,
    callbacks=[early_stop] # Utiliser le même early stopping
)

# Évaluation initiale de VGG16
loss_vgg16, acc_vgg16 = model_vgg16.evaluate(test_ds)
print(f"\nTest accuracy (VGG16) : {acc_vgg16:.2%}")
```

```
Epoch 1/10
582/582 ————— 101s 165ms/step - accuracy: 0.7374 - loss: 0.5659 - val_accuracy: 0.8641 - val_loss: 0
Epoch 2/10
582/582 ————— 90s 154ms/step - accuracy: 0.8148 - loss: 0.4557 - val_accuracy: 0.8801 - val_loss: 0.
Epoch 3/10
582/582 ————— 90s 154ms/step - accuracy: 0.8267 - loss: 0.4156 - val_accuracy: 0.8878 - val_loss: 0.
73/73 ————— 10s 134ms/step - accuracy: 0.8719 - loss: 0.4189

Test accuracy (VGG16) : 87.19%
```

✓ Évaluation détaillée (Précision, Rappel, F1-score, ROC, AUC) pour VGG16

```
from sklearn.metrics import classification_report, roc_curve, auc
import numpy as np

# Obtenir les vraies étiquettes et les prédictions pour VGG16
y_true_vgg16 = []
y_pred_proba_vgg16 = []

for images, labels in test_ds:
    y_true_vgg16.extend(labels.numpy())
    y_pred_proba_vgg16.extend(model_vgg16.predict(images, verbose=0).flatten())

y_true_vgg16 = np.array(y_true_vgg16)
y_pred_proba_vgg16 = np.array(y_pred_proba_vgg16)

# Convertir les probabilités en prédictions binaires (seuil 0.5)
y_pred_vgg16 = (y_pred_proba_vgg16 > 0.5).astype(int)

print("\n--- Rapport de Classification (VGG16) ---")
print(classification_report(y_true_vgg16, y_pred_vgg16, target_names=['Chat', 'Chien']))

# Calculer la courbe ROC et l'AUC pour VGG16
fpr_vgg16, tpr_vgg16, thresholds_vgg16 = roc_curve(y_true_vgg16, y_pred_proba_vgg16)
roc_auc_vgg16 = auc(fpr_vgg16, tpr_vgg16)
```

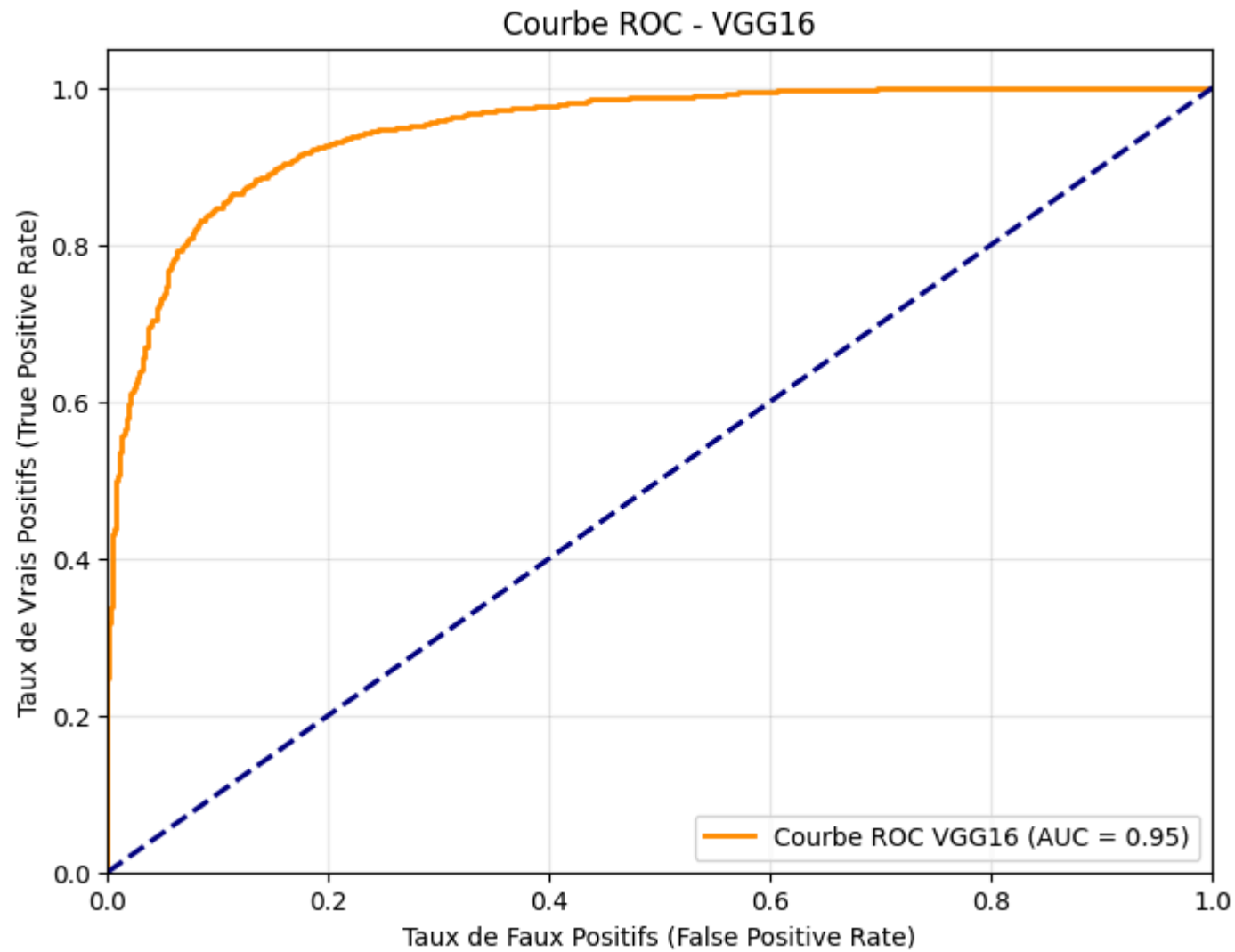
```
print(f"AUC (VGG16): {roc_auc_vgg16:.4f}")
```

--- Rapport de Classification (VGG16) ---

	precision	recall	f1-score	support
Chat	0.89	0.85	0.87	1160
Chien	0.86	0.89	0.87	1166
accuracy			0.87	2326
macro avg	0.87	0.87	0.87	2326
weighted avg	0.87	0.87	0.87	2326

AUC (VGG16): 0.9478

```
fig_vgg16 = plt.figure(figsize=(8, 6))
plt.plot(fpr_vgg16, tpr_vgg16, color='darkorange', lw=2, label=f'Courbe ROC VGG16 (AUC = {roc_auc_vgg16:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('Taux de Faux Positifs (False Positive Rate)')
plt.ylabel('Taux de Vrais Positifs (True Positive Rate)')
plt.title('Courbe ROC - VGG16')
plt.legend(loc='lower right')
plt.grid(True, alpha=0.3)
plt.show()
```



✓ step_artifacts

```
# @title step_artifacts
num_fig = "4" # @param {type:"string"}
step = 'ModelEvaluation' # @param ["DataLoading", "I
```

num_fig

4

step

ModelEvaluation

```
# Assuming `fig_vgg16` is the current matplotlib fig
```

[Masquer le code](#)

- ✓ Implémentation et Évaluation de ResNet50
- ✓ Construction du modèle ResNet50